



# # Competitive Security Assessment

fishcake

Sep 15th, 2025



|                                                                                                                           |    |
|---------------------------------------------------------------------------------------------------------------------------|----|
| Summary                                                                                                                   | 3  |
| Overview                                                                                                                  | 4  |
| Audit Scope                                                                                                               | 5  |
| Code Assessment Findings                                                                                                  | 6  |
| FIC-1 <code>nftUpgradeInit</code> can never be called                                                                     | 9  |
| FIC-2 <code>fccAddress</code> address is left uninitialized DoSing all withdrawal and deposits                            | 11 |
| FIC-3 Users can be assigned to the highest-tier NFT (type 6) due to incorrect <code>mineAmount</code> validation          | 13 |
| FIC-4 Users Can Stake Without Paying Tokens                                                                               | 15 |
| FIC-5 Users Can Repeatedly Withdraw Funds From <code>StakingManager</code>                                                | 17 |
| FIC-6 NFT ownership logic errors in <code>NftManagerV5::mintBoosterNFT</code>                                             | 20 |
| FIC-7 Incorrect <code>decimal</code> precision in <code>NftManagerV5::mintBoosterNFT()</code>                             | 22 |
| FIC-8 <code>DirectSalePoolV1::buyFccAmount</code> function does not check if <code>payUsdtAmount</code> is greater than 0 | 24 |
| FIC-9 Use <code>safeTransferFrom</code> instead of <code>transferFrom</code>                                              | 26 |
| FIC-10 Tiered Pricing System Vulnerable to Order Aggregation Exploitation                                                 | 29 |
| FIC-11 Incorrect <code>_dropNumber</code> validation allows increasing mined rewards                                      | 31 |
| FIC-12 Inconsistent Token Burn Tracking Due to Multiple Burn Mechanisms                                                   | 33 |
| FIC-13 Improper use of <code>initializer</code> modifier in internal storage initialization functions                     | 35 |
| FIC-14 Hardcoded USDT Decimal Value Leading to Cross-Chain Incompatibility                                                | 37 |
| FIC-15 <code>DepositIntoStaking</code> function does not verify whether the Booster NFT exists.                           | 40 |
| FIC-16 Any ETH received on <code>stakingManager</code> would locked permanently.                                          | 42 |
| FIC-17 Uninitialized Economic Parameters Allow Free NFT Minting                                                           | 43 |
| FIC-18 The <code>InvestorSalePool</code> contract lacks function to withdraw FCC tokens from the contract.                | 44 |
| FIC-19 Inconsistent Error Message in User Type Validation Leading to Logic Confusion                                      | 45 |
| FIC-20 Redundant Code                                                                                                     | 50 |
| FIC-21 Missing storage gap for upgradeable contract                                                                       | 52 |
| FIC-22 Missing <code>ReentrancyGuardUpgradeable</code> Initialization                                                     | 53 |
| FIC-23 Missing Event                                                                                                      | 55 |
| FIC-24 Incorrect minimum stake validation                                                                                 | 56 |
| FIC-25 Incorrect error message in <code>StakingManager::getStakingPeriodAndApr()</code>                                   | 57 |
| FIC-26 Incorrect Upgradeable Imports Break Initialization Protection                                                      | 58 |
| FIC-27 Function Naming Convention Violation                                                                               | 59 |
| Disclaimer                                                                                                                | 60 |

## Summary

This report is prepared for the project to identify vulnerabilities and issues in the smart contract source code. A group of NDA covered experienced security experts have participated in the Secure3's Audit Contest to find vulnerabilities and optimizations. Secure3 team has participated in the contest process as well to provide extra auditing coverage and scrutiny of the finding submissions.

The comprehensive examination and auditing scope includes:

- Cross checking contract implementation against functionalities described in the documents and white paper disclosed by the project owner.
- Contract Privilege Role Review to provide more clarity on smart contract roles and privilege.
- Using static analysis tools to analyze smart contracts against common known vulnerabilities patterns.
- Verify the code base is compliant with the most up-to-date industry standards and security best practices.
- Comprehensive line-by-line manual code review of the entire codebase by industry experts.

The security assessment resulted in findings that are categorized in four severity levels: Critical, Medium, Low, Informational. For each of the findings, the report has included recommendations of fix or mitigation for security and best practices.

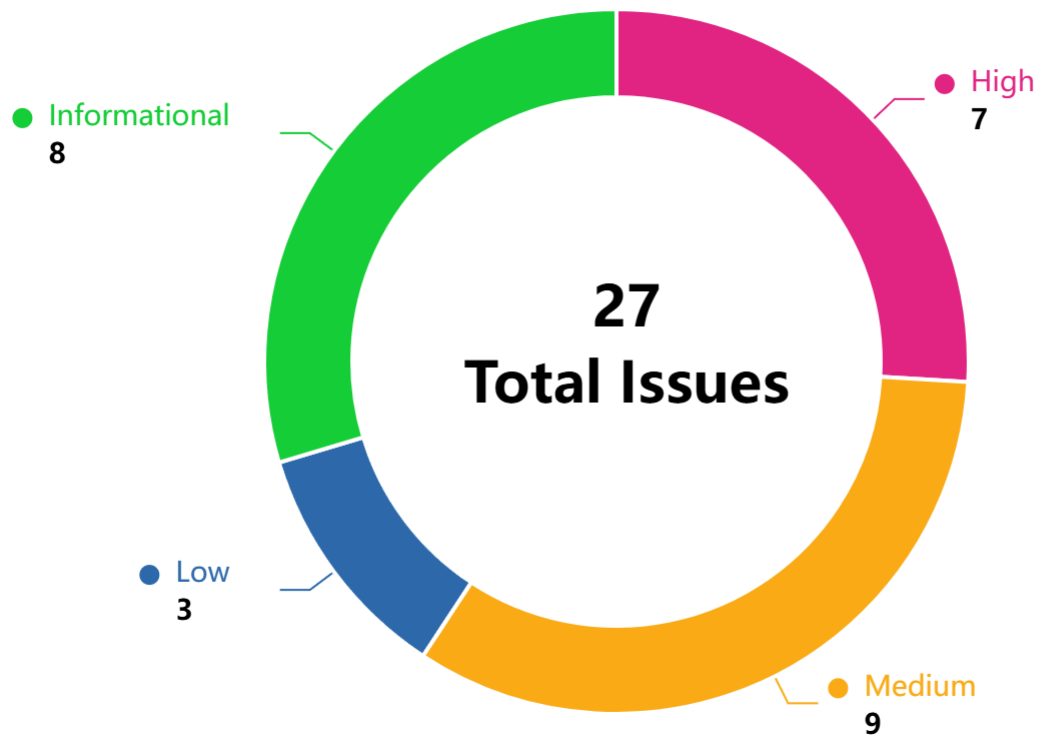
## Overview

|              |                                                                                                                                                                                                                                                                                                       |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Project Name | fishcake                                                                                                                                                                                                                                                                                              |
| Language     | solidity                                                                                                                                                                                                                                                                                              |
| Codebase     | <ul style="list-style-type: none"><li>• <a href="https://github.com/FishcakeLab/fishcake-contracts">https://github.com/FishcakeLab/fishcake-contracts</a></li><li>• audit version-6339ddeddd6143cf1916f8aa6092d97b816411a7</li><li>• final version-5479fe0b4663840cd06029eb5ae56ddbda8ecfc6</li></ul> |

# Audit Scope

| File                                                | SHA256 Hash                                                      |
|-----------------------------------------------------|------------------------------------------------------------------|
| src/contracts/core/token/NftManagerV5.sol           | 59d67d1692c6d8ceb8996167632d9fa1dec67212339397d8351579e705340c77 |
| src/contracts/core/FishcakeEventManagerV2.sol       | 16a45d83ad243647d34b0e1e33a3f3ae13326c90e1ae047970d1684069801965 |
| src/contracts/core/StakingManager.sol               | cc61d82da84b2e00c2d1733dba072a6ef113fe53ffd1ee82b0b50811452a7a6d |
| src/contracts/core/sale/InvestorSalePool.sol        | 769d8209d332f8cf92771e4959088a5d142a5a84f4ff7d2f307de0446d931412 |
| src/contracts/core/token/FishCakeCoin.sol           | 2b11bc5c1511e9d17b99ed0db8ce13e9b4707543985b78015408b0b51a4445d4 |
| src/contracts/core/FishcakeEventManagerStorage.sol  | cd77e9b01405433eb8b2aa85711d35ca3ce0262982d51ce42c22db557f5e0900 |
| src/contracts/core/sale/DirectSalePoolV1.sol        | 5a5b2003a967ce170b30765be013f2698a36a2d1fada1c61c8ff67f3a392a6cb |
| src/contracts/core/token/NftManagerStorage.sol      | 20baa4cf7610fe737794f80d53fa4e1bd7ef94a4c5a4371ebf86add8a69a5346 |
| src/contracts/core/RedemptionPool.sol               | e9e64e0a2023a7fab767712c0a651a20bc572a2c4b0fbf7fefc73543348d6bbc |
| src/contracts/core/StakingManagerStorage.sol        | 9916eab432b2158ca22a34034b58729f600f95620090f45351ab378a90f2c91c |
| src/contracts/core/sale/InvestorSalePoolStorage.sol | d68c36fb4fb60f4e6cae6100214b27b7626d58f908780bff262fa521a36c004f |
| src/contracts/core/token/FishCakeCoinStorage.sol    | 29a7bae981f9438ebf44bab76d85a2848d3594e8ff3411446cc94727a19ac56a |
| src/contracts/core/sale/DirectSalePoolStorage.sol   | a6de67840c7719df25f767f611ea26856d90c5053f8a86662621022480036242 |

# Code Assessment Findings



| ID    | Name                                                                                                       | Category | Severity | Client Response | Contributor |
|-------|------------------------------------------------------------------------------------------------------------|----------|----------|-----------------|-------------|
| FIC-1 | <code>nftUpgradeInit</code> can never be called                                                            | Logical  | High     | Fixed           | ***         |
| FIC-2 | <code>fccAddress</code> address is left uninitialized DoSing all withdrawal and deposits                   | Logical  | High     | Fixed           | ***         |
| FIC-3 | Users can be assigned to the highest-tier NFT (type 6) due to incorrect <code>mineAmount</code> validation | Logical  | High     | Fixed           | ***         |
| FIC-4 | Users Can Stake Without Paying Tokens                                                                      | DOS      | High     | Fixed           | ***         |
| FIC-5 | Users Can Repeatedly Withdraw Funds From <code>StakingManager</code>                                       | Logical  | High     | Fixed           | ***         |
| FIC-6 | NFT ownership logic errors in <code>NftManagerV5::mintBoosterNFT</code>                                    | Logical  | High     | Fixed           | ***         |

|        |                                                                                                                     |                   |        |              |     |
|--------|---------------------------------------------------------------------------------------------------------------------|-------------------|--------|--------------|-----|
| FIC-7  | Incorrect <code>decimal</code> precision in <code>NftManagerV5::mintBoosterNFT()</code>                             | Logical           | High   | Fixed        | *** |
| FIC-8  | <code>DirectSalePoolV1::buyFccAmount</code> function does not check if <code>payUsdtAmount</code> is greater than 0 | Logical           | Medium | Fixed        | *** |
| FIC-9  | Use <code>safeTransferFrom</code> instead of <code>transferFrom</code>                                              | Logical           | Medium | Fixed        | *** |
| FIC-10 | Tiered Pricing System Vulnerable to Order Aggregation Exploitation                                                  | Logical           | Medium | Fixed        | *** |
| FIC-11 | Incorrect <code>_dropNumber</code> validation allows increasing mined rewards                                       | Logical           | Medium | Fixed        | *** |
| FIC-12 | Inconsistent Token Burn Tracking Due to Multiple Burn Mechanisms                                                    | Logical           | Medium | Fixed        | *** |
| FIC-13 | Improper use of <code>initialize</code> modifier in internal storage initialization functions                       | Logical           | Medium | Fixed        | *** |
| FIC-14 | Hardcoded USDT Decimal Value Leading to Cross-Chain Incompatibility                                                 | Logical           | Medium | Acknowledged | *** |
| FIC-15 | <code>DepositIntoStaking</code> function does not verify whether the Booster NFT exists.                            | Logical           | Medium | Fixed        | *** |
| FIC-16 | Any ETH received on stakingManager would be locked permanently.                                                     | Language Specific | Medium | Fixed        | *** |
| FIC-17 | Uninitialized Economic Parameters Allow Free NFT Minting                                                            | Logical           | Low    | Fixed        | *** |
| FIC-18 | The <code>InvestorSalePool</code> contract lacks function to withdraw FCC tokens from the contract.                 | Logical           | Low    | Fixed        | *** |

|        |                                                                                  |            |               |       |     |
|--------|----------------------------------------------------------------------------------|------------|---------------|-------|-----|
| FIC-19 | Inconsistent Error Message in User Type Validation Leading to Logic Confusion    | Logical    | Low           | Fixed | *** |
| FIC-20 | Redundant Code                                                                   | Logical    | Informational | Fixed | *** |
| FIC-21 | Missing storage gap for upgradeable contract                                     | Logical    | Informational | Fixed | *** |
| FIC-22 | Missing ReentrancyGuardUpgradeable Initialization                                | Logical    | Informational | Fixed | *** |
| FIC-23 | Missing Event                                                                    | Code Style | Informational | Fixed | *** |
| FIC-24 | Incorrect minimum stake validation                                               | Logical    | Informational | Fixed | *** |
| FIC-25 | Incorrect error message in <code>stakingManager::getStakingPeriodAndApr()</code> | Logical    | Informational | Fixed | *** |
| FIC-26 | Incorrect Upgradeable Imports Break Initialization Protection                    | Logical    | Informational | Fixed | *** |
| FIC-27 | Function Naming Convention Violation                                             | Logical    | Informational | Fixed | *** |



# FIC-1: `nftUpgradeInit` can never be called

| Category | Severity | Client Response | Contributor |
|----------|----------|-----------------|-------------|
| Logical  | High     | Fixed           | ***         |

## Code Reference

- code/src/contracts/core/token/NftManagerV5.sol#L97-L106

```
97: function nftUpgradeInit(address _feManagerAddress, address _boosterAddress, address _stakingManagerAddress) external onlyBooster {
98:     uncommonFishcakeNftJson = "https://www.fishcake.org/image/3.json";
99:     rareShrimpNftJson = "https://www.fishcake.org/image/4.json";
100:    epicSalmonNftJson = "https://www.fishcake.org/image/5.json";
101:    legendaryTunaNftJson = "https://www.fishcake.org/image/6.json";
102:
103:    feManagerAddress = IFishcakeEventManager(_feManagerAddress);
104:    stakingManagerAddress = IStakingManager(_stakingManagerAddress);
105:    boosterAddress = _boosterAddress;
106: }
```

## Description

\*\*\*: The `NftManagerV5` contract contains a critical circular dependency in its initialization logic:

- a) The `nftUpgradeInit` function requires the caller to be `boosterAddress` (via `onlyBooster`).
- b) However, `boosterAddress` is uninitialized (`address(0)`) until `nftUpgradeInit` sets it.

This creates an unresolvable deadlock:

- a) `nftUpgradeInit` cannot be called because `boosterAddress` is not set.
- b) `boosterAddress` cannot be set because `nftUpgradeInit` is the only function that assigns it.

The `onlyBooster` modifier logic:

```
modifier onlyBooster() {
    require(
        msg.sender == boosterAddress,
        "MessageManager: only booster address can do this operate"
    );
    _;
}
```

This check fails if `boosterAddress` is `address(0)` (default state).

The `nftUpgradeInit` Function:

```
function nftUpgradeInit(address _feManagerAddress, address _boosterAddress, address _stakingManagerAddress)
external onlyBooster {
    uncommonFishcakeNftJson = "https://www.fishcake.org/image/3.json";
    rareShrimpNftJson = "https://www.fishcake.org/image/4.json";
    epicSalmonNftJson = "https://www.fishcake.org/image/5.json";
    legendaryTunaNftJson = "https://www.fishcake.org/image/6.json";

    feManagerAddress = IFishcakeEventManager(_feManagerAddress);
    stakingManagerAddress = IStakingManager(_stakingManagerAddress);
    boosterAddress = _boosterAddress;
}
```

The function cannot be called until `boosterAddress` is set, but `boosterAddress` is set inside the function.

## Recommendation

\*\*\*: Consider initializing `boosterAddress` in `initialize()`:

```
function initialize(
    address _initialOwner,
    address _fccTokenAddr,
    address _tokenUsdtAddr,
    address _redemptionPoolAddress,
    address _boosterAddress // <-- New parameter
) public initializer {
    // ... existing code ...
    boosterAddress = _boosterAddress; // <-- Set here
}
```

## Client Response

client response : Fixed.

## FIC-2: fccAddress address is left uninitialized DoSing all withdrawal and deposits

| Category | Severity | Client Response | Contributor |
|----------|----------|-----------------|-------------|
| Logical  | High     | Fixed           | ***         |

### Code Reference

- code/src/contracts/core/StakingManagerStorage.sol#L33

```
33: address public fccAddress;
```

### Description

\*\*\*: fccAddress acts as a token used during deposit and withdrawal.

```
function DepositIntoStaking(uint256 amount, uint8 stakingType) external nonReentrant {
    require(amount > minStakeAmount, "StakingManager DepositIntoStaking: staking amount must be more than minStakeAmount");

    IERC20(fccAddress).safeTransfer(address(this), amount); // @audit this should be transferFrom right?

    bytes32 txMessageHash = keccak256(
        abi.encode(
            msg.sender,
            fccAddress,
            amount,
            messageNonce
        )
    );
    ...snipped for brevity...
}
```

```
function withdrawFromStakingWithAprIncome(uint256 amount, uint256 messageNonce) external nonReentrant {
    bytes32 txMessageHash = keccak256(
        abi.encode(
            msg.sender,
            fccAddress,
            amount,
            messageNonce
        )
    );
    ...snipped for brevity...

    @> IERC20(fccAddress).safeTransfer(msg.sender, amount);
    @> IERC20(fccAddress).safeTransfer(msg.sender, rewardAprFunding);

}
```

The issue is that fccAddress is left uninitialized in the contract, resulting to an unreachable token address.

## Impact

Uninitialized FccAddress would DoS critical protocol functionality

## Recommendation

\*\*\*: Initialize fccAddress during initialization

## Client Response

client response : Fixed.

## FIC-3:Users can be assigned to the highest-tier NFT (type 6) due to incorrect `mineAmount` validation

| Category | Severity | Client Response | Contributor |
|----------|----------|-----------------|-------------|
| Logical  | High     | Fixed           | ***         |

### Code Reference

- code/src/contracts/core/token/NftManagerV5.sol#L108-L128

```
108: function mintBoosterNFT(address miner) external onlyBooster nonReentrant returns (bool, uint256) {
109:     uint256 mineAmount = feManagerAddress.getMinerMineAmount(miner);
110:     if (mineAmount < 30) {
111:         revert MineAmountNotEnough(mineAmount);
112:     }
113:     uint256 boosterTokenId = _nextTokenId++;
114:     _safeMint(msg.sender, boosterTokenId);
115:     uint256 decimal = 10e6;
116:     if(mineAmount >= 30 * decimal && mineAmount < 90 * decimal) {
117:         nftMintType[boosterTokenId] = 3;
118:     } else if(mineAmount >= 90 * decimal && mineAmount < 300 * decimal) {
119:         nftMintType[boosterTokenId] = 4;
120:     } else if(mineAmount >= 300 * decimal && mineAmount < 900 * decimal) {
121:         nftMintType[boosterTokenId] = 5;
122:     } else {
123:         nftMintType[boosterTokenId] = 6;
124:     }
125:     feManagerAddress.deleteMinerMineAmount(miner);
126:     minerActiveNft[miner] = boosterTokenId;
127:     return (true, boosterTokenId);
128: }
```

### Description

\*\*\*: The `mintBoosterNFT` function contains a critical logic error in its `mineAmount` validation that incorrectly assigns the highest-tier NFT (type 6) to users who only meet the minimum requirement ( $\geq 30$  tokens) rather than the intended maximum requirement ( $\geq 900e6$  tokens). This occurs due to improper decimal handling in the initial check and flawed conditional logic in the tier assignment.

The insufficient initial validation in `mintBoosterNFT` function:

```
uint256 mineAmount = feManagerAddress.getMinerMineAmount(miner);
if (mineAmount < 30) {
    revert MineAmountNotEnough(mineAmount);
}
```

It only checks against raw value `30` without considering decimals. The correct one should be `30 ** 10e6`. The flawed tier assignment logic:

```
if(mineAmount >= 30 * decimal && mineAmount < 90 * decimal) {
    nftMintType[boosterTokenId] = 3;
} else if(mineAmount >= 90 * decimal && mineAmount < 300 * decimal) {
    nftMintType[boosterTokenId] = 4;
} else if(mineAmount >= 300 * decimal && mineAmount < 900 * decimal) {
    nftMintType[boosterTokenId] = 5;
} else {
    nftMintType[boosterTokenId] = 6;
}
```

The final `else` clause acts as a catch-all for any amount not caught by previous conditions. This includes the range `30 ≤ mineAmount < 30e6`, incorrectly assigning type `6`.

## Recommendation

\*\*\*: Consider following fix:

```
function mintBoosterNFT(address miner) external onlyBooster nonReentrant returns (bool, uint256) {
    uint256 mineAmount = feManagerAddress.getMineAmount(miner);
    uint256 decimal = 10e6;

    // Proper decimal-aware validation
    if (mineAmount < 30 * decimal) {
        revert MineAmountNotEnough(mineAmount);
    }
    ...
    ...
}
```

## Client Response

client response : Fixed.

## FIC-4:Users Can Stake Without Paying Tokens

| Category | Severity | Client Response | Contributor |
|----------|----------|-----------------|-------------|
| DOS      | High     | Fixed           | ***         |

### Code Reference

- code/src/contracts/core/StakingManager.sol#L39-L78

```
39: function DepositIntoStaking(uint256 amount, uint8 stakingType) external nonReentrant {
40:     require(amount > minStakeAmount, "StakingManager DepositIntoStaking: staking amount must be more t
han minStakeAmount");
41:
42:     IERC20(fccAddress).safeTransfer(address(this), amount);
43:
44:     bytes32 txMessageHash = keccak256(
45:         abi.encode(
46:             msg.sender,
47:             fccAddress,
48:             amount,
49:             messageNonce
50:         )
51:     );
52:     uint256 stakingTimestamp = 0;
53:     uint256 apr = 0;
54:     (stakingTimestamp, apr) = getStakingPeriodAndApr(stakingType);
55:     uint endTime = block.timestamp + stakingTimestamp;
56:
57:     uint256 tokenId = nftManagerAddress.getActiveMinerBoosterNft(msg.sender);
58:
```

```
59:     stakeHolderStakingInfo memory ssInfo = stakeHolderStakingInfo({
60:         startStakingTime: block.timestamp,
61:         amount: amount,
62:         messageNonce: messageNonce,
63:         endStakingTime: endTime,
64:         stakingStatus: 0, // under staking now
65:         stakingType: stakingType,
66:         bindingNft: tokenId
67:     });
68:
69:     nftManagerAddress.inActiveMinerBoosterNft(msg.sender);
70:
71:     stakingQueued[msg.sender][txMessageHash] = ssInfo;
72:
73:     totalStakingAmount += amount;
74:
75:     emit StakeHolderDepositStaking(msg.sender, amount, messageNonce);
76:
77:     messageNonce++;
78: }
```

### Description

\*\*\*: The `DepositIntoStaking` function in the `StakingManager` contract contains a critical security flaw where it incorrectly uses `safeTransfer` instead of `safeTransferFrom` to move user funds. This allows users to stake tokens

without actually transferring them, effectively enabling "free staking" and breaking the core economic logic of the contract.

The current implementation is:

```
function DepositIntoStaking(uint256 amount, uint8 stakingType) external nonReentrant {
    require(amount > minStakeAmount, "StakingManager DepositIntoStaking: staking amount must be more than minStakeAmount");

    IERC20(fccAddress).safeTransfer(address(this), amount);

    bytes32 txMessageHash = keccak256(
        abi.encode(
            msg.sender,
            fccAddress,
            amount,
            messageNonce
        )
    );
    ...
    ...
}
```

Here the `safeTransfer` moves tokens from the `contract itself` (`address(this)`) to `address(this)`. Since the contract is transferring its own tokens (if any), users are not required to deposit any funds. This makes staking effectively free, as the contract does not deduct tokens from the user's wallet.

## Recommendation

\*\*\*: Consider following fix:

```
function DepositIntoStaking(uint256 amount, uint8 stakingType) external nonReentrant {
    require(amount > minStakeAmount, "StakingManager DepositIntoStaking: staking amount must be more than minStakeAmount");
    IERC20(fccAddress).safeTransferFrom(msg.sender, address(this), amount);
    ...
    ...
}
```

## Client Response

client response : Fixed.



# FIC-5:Users Can Repeatedly Withdraw Funds From StakingManager

| Category | Severity | Client Response | Contributor |
|----------|----------|-----------------|-------------|
| Logical  | High     | Fixed           | ***         |

## Code Reference

- code/src/contracts/core/StakingManager.sol#L80-L115

```
80: function withdrawFromStakingWithAprIncome(uint256 amount, uint256 messageNonce) external nonReentrant {
81:     bytes32 txMessageHash = keccak256(
82:         abi.encode(
83:             msg.sender,
84:             fccAddress,
85:             amount,
86:             messageNonce
87:         )
88:     );
89:     uint256 txLockEndTime = stakingQueued[msg.sender][txMessageHash].endStakingTime;
90:     if (block.timestamp < txLockEndTime) {
91:         revert FundingUnderStaking(
92:             amount,
93:             txLockEndTime
94:         );
95:     }
96:     uint256 amountOut = stakingQueued[msg.sender][txMessageHash].amount;
97:     if (amountOut < minStakeAmount) {
98:         revert NoFundingForStaking();
99:     }
```

```
100:     totalStakingAmount -= amount;
101:     stakingQueued[msg.sender][txMessageHash].stakingStatus = 1; //staking end
102:
103:     uint256 rewardAprFunding = calculateAprFunding(
104:         msg.sender,
105:         stakingQueued[msg.sender][txMessageHash].amount,
106:         stakingQueued[msg.sender][txMessageHash].stakingType,
107:         stakingQueued[msg.sender][txMessageHash].startStakingTime,
108:         stakingQueued[msg.sender][txMessageHash].bindingNft
109:     );
110:
111:     IERC20(fccAddress).safeTransfer(msg.sender, amount);
112:     IERC20(fccAddress).safeTransfer(msg.sender, rewardAprFunding);
113:
114:     emit StakeHolderWithdrawStaking(msg.sender, amount, messageNonce, txMessageHash);
115: }
```

## Description

\*\*\*: A critical vulnerability has been identified in the `withdrawFromStakingWithAprIncome` function of the `StakingManager` contract. The function fails to verify the `stakingStatus` of the staking position before processing withdrawals, allowing malicious users to repeatedly withdraw the same staked funds and associated rewards, potentially draining the contract's assets.

The current implementation:

```
function withdrawFromStakingWithAprIncome(uint256 amount, uint256 messageNonce) external nonReentrant {
    bytes32 txMessageHash = keccak256(
        abi.encode(
            msg.sender,
            fccAddress,
            amount,
            messageNonce
        )
    );
    uint256 txLockEndTime = stakingQueued[msg.sender][txMessageHash].endStakingTime;
    if (block.timestamp < txLockEndTime) {
        revert FundingUnderStaking(
            amount,
            txLockEndTime
        );
    }
    uint256 amountOut = stakingQueued[msg.sender][txMessageHash].amount;
    if (amountOut < minStakeAmount) {
        revert NoFundingForStaking();
    }
    totalStakingAmount -= amount;
    stakingQueued[msg.sender][txMessageHash].stakingStatus = 1; //staking end

    uint256 rewardAprFunding = calculateAprFunding(
        msg.sender,
        stakingQueued[msg.sender][txMessageHash].amount,
        stakingQueued[msg.sender][txMessageHash].stakingType,
        stakingQueued[msg.sender][txMessageHash].startStakingTime,
        stakingQueued[msg.sender][txMessageHash].bindingNft
    );

    IERC20(fccAddress).safeTransfer(msg.sender, amount);
    IERC20(fccAddress).safeTransfer(msg.sender, rewardAprFunding);

    emit StakeHolderWithdrawStaking(msg.sender, amount, messageNonce, txMessageHash);
}
```

The function only checks if the staking period has ended ( `block.timestamp >= txLockEndTime` ). While it sets `stakingStatus = 1` after withdrawal, it doesn't verify this status before processing. As a result, malicious users can repeatedly withdraw the same staked funds and associated rewards, potentially draining the contract's assets.

## Recommendation

\*\*\*: Consider adding following checking:

```
function withdrawFromStakingWithAprIncome(uint256 amount, uint256 messageNonce) external nonReentrant {
    bytes32 txMessageHash = keccak256(
        abi.encode(
            msg.sender,
            fccAddress,
            amount,
            messageNonce
        )
    );
    uint256 txLockEndTime = stakingQueued[msg.sender][txMessageHash].endStakingTime;
    if (block.timestamp < txLockEndTime) {
        revert FundingUnderStaking(
            amount,
            txLockEndTime
        );
    }
    require(stakingQueued[msg.sender][txMessageHash].stakingStatus == 0, "already withdrawn");
    ...
    ...
}
```

## Client Response

client response : Fixed.

# FIC-6:NFT ownership logic errors in NftManagerV5::mintBoosterNFT

| Category | Severity | Client Response | Contributor |
|----------|----------|-----------------|-------------|
| Logical  | High     | Fixed           | ***         |

## Code Reference

- code/src/contracts/core/token/NftManagerV5.sol#L114

```
114: _safeMint(msg.sender, boosterTokenId);
```

## Description

\*\*\*: The NFT is minted to `msg.sender` (the booster address), but setting `minerActiveNft[miner] = boosterTokenId;` on line 126 causes inconsistency between ownership and system logic.

```
function mintBoosterNFT(address miner) external onlyBooster nonReentrant returns (bool, uint256) {
    uint256 mineAmount = feManagerAddress.getMiningAmount(miner);
    if (mineAmount < 30) {
        revert MiningAmountNotEnough(mineAmount);
    }
    uint256 boosterTokenId = _nextTokenId++;
    _safeMint(msg.sender, boosterTokenId); // @audit: should be miner
    uint256 decimal = 10e6;
    if(mineAmount >= 30 * decimal && mineAmount < 90 * decimal) {
        nftMintType[boosterTokenId] = 3;
    } else if(mineAmount >= 90 * decimal && mineAmount < 300 * decimal) {
        nftMintType[boosterTokenId] = 4;
    } else if(mineAmount >= 300 * decimal && mineAmount < 900 * decimal) {
        nftMintType[boosterTokenId] = 5;
    } else {
        nftMintType[boosterTokenId] = 6;
    }
    feManagerAddress.deleteMiningAmount(miner);
    minerActiveNft[miner] = boosterTokenId;
    return (true, boosterTokenId);
}
```

Impact:

- Booster address call `mintBoosterNFT(miner)`
- The NFT is minted for the `booster`, but the system records that the `miner` owns the NFT.
- Subsequent calls to `getActiveMinerBoosterNft(miner)` return the `tokenId`, but the `miner` does not actually own the NFT.

## Recommendation

\*\*\*: `NftManagerV5::mintBoosterNFT` function should mint to miner instead of `msg.sender`:

```
diff --git a/code/src/contracts/core/token/NftManagerV5.sol b/code/src/contracts/core/token/NftManagerV5.sol
index 0b26f46..4ab0a16 100644
--- a/code/src/contracts/core/token/NftManagerV5.sol
+++ b/code/src/contracts/core/token/NftManagerV5.sol
@@ -111,7 +111,7 @@ contract NftManagerV5 is
     revert MineAmountNotEnough(mineAmount);
 }
 uint256 boosterTokenId = _nextTokenId++;
- _safeMint(msg.sender, boosterTokenId);
+ _safeMint(miner, boosterTokenId);
 uint256 decimal = 10e6;
 if(mineAmount >= 30 * decimal && mineAmount < 90 * decimal) {
     nftMintType[boosterTokenId] = 3;
```

## Client Response

client response : Fixed.

# FIC-7:Incorrect decimal precision in NftManagerV5::mintBoosterNFT()

| Category | Severity | Client Response | Contributor |
|----------|----------|-----------------|-------------|
| Logical  | High     | Fixed           | ***         |

## Code Reference

- code/src/contracts/core/token/NftManagerV5.sol#L115

```
115: uint256 decimal = 10e6;
```

## Description

\*\*\*: Fcc token has 6 decimals as defined in [InvestorSalePoolStorage](#)

```
uint256 public constant fccDecimal = 10 ** 6;
```

This is equivalent to 1e6.

Now in [mintBoosterNFT\(\)](#), the following is done:

```
uint256 mineAmount = feManagerAddress.getMineAmount(miner);
//..
uint256 boosterTokenId = _nextTokenId++;
_safeMint(msg.sender, boosterTokenId);

>> uint256 decimal = 10e6; //@audit
if(mineAmount >= 30 * decimal && mineAmount < 90 * decimal) {
    nftMintType[boosterTokenId] = 3;
} else if(mineAmount >= 90 * decimal && mineAmount < 300 * decimal) {
    nftMintType[boosterTokenId] = 4;
} else if(mineAmount >= 300 * decimal && mineAmount < 900 * decimal) {
    nftMintType[boosterTokenId] = 5;
} else {
    nftMintType[boosterTokenId] = 6;
}
```

As seen, the function uses `decimal = 10e6` instead of `1e6` which makes it 7 decimals. This is incorrect as all the threshold checks are 10x higher than they should be.

As such, users with enough `mineAmount` end up with lower `nftMintType` when actually they deserve higher types as they need 10x more `mineAmount` to qualify for each NFT tier than intended.

## Recommendation

\*\*\*: Modify

```
- uint256 decimal = 10e6;  
+ uint256 decimal = 1e6;
```

## Client Response

client response : Fixed.

## FIC-8: DirectSalePoolV1::buyFccAmount function does not check if payUsdtAmount is greater than 0

| Category | Severity | Client Response | Contributor |
|----------|----------|-----------------|-------------|
| Logical  | Medium   | Fixed           | ***         |

### Code Reference

- code/src/contracts/core/sale/DirectSalePoolV1.sol#L41

```
41: uint256 payUsdtAmount = fccAmount / 10;
```

### Description

\*\*\*: DirectSalePoolV1::buyFccAmount function is implemented as follows:

```
function buyFccAmount(uint256 fccAmount) external {
    require(fishCakeCoin.balanceOf(address(this)) >= fccAmount, "DirectSalePool buyFccAmount: fcc token is not enough");
    uint256 payUsdtAmount = fccAmount / 10; // @audit: no check if payUsdtAmount == 0
    if (tokenUsdtAddress.balanceOf(msg.sender) < payUsdtAmount) {
        revert TokenUsdtBalanceNotEnough();
    }

    totalSellFccAmount += fccAmount;
    totalReceiveUsdtAmount += payUsdtAmount;

    tokenUsdtAddress.transferFrom(msg.sender, address(redemptionPool), payUsdtAmount);

    fishCakeCoin.transfer(msg.sender, fccAmount);

    emit BuyFishcakeCoin(msg.sender, payUsdtAmount, fccAmount);
}
```

payUsdtAmount is the amount of USDT the user needs to pay, and fccAmount is the number of FCC tokens the user wants to purchase. Due to rounding errors, when fccAmount is 9, the calculated amount of USDT the user needs to pay is 0. Because there is no check to ensure that payUsdtAmount must be greater than 0, the user can get 9 FCC tokens for free.

### Recommendation

\*\*\*:



```
diff --git a/code/src/contracts/core/sale/DirectSalePoolV1.sol b/code/src/contracts/core/sale/DirectSalePoolV1.sol
index bff8b64..bda82ee 100644
--- a/code/src/contracts/core/sale/DirectSalePoolV1.sol
+++ b/code/src/contracts/core/sale/DirectSalePoolV1.sol
@@ -39,6 +39,7 @@ contract DirectSalePoolV1 is Initializable, ERC20Upgradeable, ERC20BurnableUpgra
    function buyFccAmount(uint256 fccAmount) external {
        require(fishCakeCoin.balanceOf(address(this)) >= fccAmount, "DirectSalePool buyFccAmount: fcc token
is not enough");
        uint256 payUsdtAmount = fccAmount / 10;
+       require(payUsdtAmount > 0, "DirectSalePool buyFccAmount: payUsdtAmount is zero");
        if (tokenUsdtAddress.balanceOf(msg.sender) < payUsdtAmount) {
            revert TokenUsdtBalanceNotEnough();
        }
    }
```

## Client Response

client response : Fixed.

## FIC-9: Use `safeTransferFrom` instead of `transferFrom`

| Category | Severity | Client Response | Contributor |
|----------|----------|-----------------|-------------|
| Logical  | Medium   | Fixed           | ***         |

### Code Reference

- code/src/contracts/core/sale/DirectSalePoolV1.sol#L49
- code/src/contracts/core/sale/DirectSalePoolV1.sol#L67

```
49: tokenUsdtAddress.transferFrom(msg.sender, address(redemptionPool), payUsdtAmount);
```

```
67: tokenUsdtAddress.transferFrom(msg.sender, address(redemptionPool), tokenUsdtAmount);
```

- code/src/contracts/core/sale/InvestorSalePool.sol#L70-L71

```
70: tokenUsdtAddress.transferFrom(msg.sender, address(this), tokenUsdtAmount / 2);  
71:     tokenUsdtAddress.transferFrom(msg.sender, address(redemptionPool), tokenUsdtAmount / 2);
```

### Description

\*\*\*: In the `DirectSalePoolV1` contract, the `buyFccAmount` function and `buyFccByUsdtAmount` code are as follows:

```
function buyFccAmount(uint256 fccAmount) external {
    require(fishCakeCoin.balanceOf(address(this)) >= fccAmount, "DirectSalePool buyFccAmount: fcc token
is not enough");
    uint256 payUsdtAmount = fccAmount / 10;
    if (tokenUsdtAddress.balanceOf(msg.sender) < payUsdtAmount) {
        revert TokenUsdtBalanceNotEnough();
    }

    totalSellFccAmount += fccAmount;
    totalReceiveUsdtAmount += payUsdtAmount;

    tokenUsdtAddress.transferFrom(msg.sender, address(redemptionPool), payUsdtAmount); // @audit: use sa
feTransferFrom

    fishCakeCoin.transfer(msg.sender, fccAmount);

    emit BuyFishcakeCoin(msg.sender, payUsdtAmount, fccAmount);
}

function buyFccByUsdtAmount(uint256 tokenUsdtAmount) external {
    require(tokenUsdtAddress.balanceOf(msg.sender) >= tokenUsdtAmount, "DirectSalePool buyFccAmount: usd
t token is not enough");
    uint256 sellFccAmount = tokenUsdtAmount * 10; // 1 USDT = 10 FCC
    if (sellFccAmount > fishCakeCoin.balanceOf(address(this))) {
        revert FishcakeTokenNotEnough();
    }

    totalSellFccAmount += sellFccAmount;
    totalReceiveUsdtAmount += tokenUsdtAmount;

    tokenUsdtAddress.transferFrom(msg.sender, address(redemptionPool), tokenUsdtAmount); // @audit: use
safeTransferFrom

    fishCakeCoin.transfer(msg.sender, sellFccAmount);

    emit BuyFishcakeCoin(msg.sender, tokenUsdtAmount, sellFccAmount);
}
```

`tokenUsdtAddress.transferFrom` is used to transfer the ERC20 token (USDT) to the contract.

However, `transfer` and `transferFrom` functions return a boolean value indicating success. This parameter needs to be checked for success. Some tokens do not revert if the transfer failed but return false instead.

`USDT` doesn't correctly implement EIP20 and their `transfer/transferFrom` function returns void instead of success boolean: <https://etherscan.io/address/0xdac17f958d2ee523a2206206994597c13d831ec7#code#L126>

In this case, `DirectSalePoolV1::buyFccAmount` and `DirectSalePoolV1::buyFccByUsdtAmount` cannot transfer `USDT`.

## Recommendation

\*\*\*: Recommend using SafeERC20 library from OpenZeppelin:

<https://github.com/OpenZeppelin/openzeppelin-contracts/blob/master/contracts/token/ERC20/utils/SafeERC20.sol#L41-L43>

## Client Response

client response : Fixed.

# FIC-10: Tiered Pricing System Vulnerable to Order Aggregation Exploitation

| Category | Severity | Client Response | Contributor |
|----------|----------|-----------------|-------------|
| Logical  | Medium   | Fixed           | ***         |

## Code Reference

- code/src/contracts/core/sale/InvestorSalePool.sol#L92-L107

```
92: function calculateFccByUsdt(uint256 _usdtAmount) internal pure returns (uint256) {
93:     if (_usdtAmount >= 100_000 * usdtDecimal) { // Tier 1: 1 FCC = 0.06 USDT
94:         return (_usdtAmount * 100 * fccDecimal) / (6 * usdtDecimal);
95:     } else if (_usdtAmount < 100_000 * usdtDecimal && _usdtAmount >= 10_000 * usdtDecimal) { // Tier
2: 1 FCC = 0.07 USDT
96:         return (_usdtAmount * 100 * fccDecimal) / (7 * usdtDecimal);
97:     } else if (_usdtAmount < 10_000 * usdtDecimal && _usdtAmount >= 5_000 * usdtDecimal) { // Tier 3:
1 FCC = 0.08 USDT
98:         return (_usdtAmount * 100 * fccDecimal) / (8 * usdtDecimal);
99:     } else if (_usdtAmount < 5_000 * usdtDecimal && _usdtAmount >= 1_000 * usdtDecimal) { // Tier 4: 1
FCC = 0.09 USDT
100:         return (_usdtAmount * 100 * fccDecimal) / (9 * usdtDecimal);
101:     } else if (_usdtAmount < 1_000 * usdtDecimal && _usdtAmount > 0 * usdtDecimal) { // Tier 5: 1 FCC
= 0.1 USDT
102:         return (_usdtAmount * 10 * fccDecimal) / usdtDecimal;
103:     } else {
104:         revert NotSupportUsdtAmount();
105:     }
106: }
```

## Description

\*\*\*: The contract uses a tiered pricing system where larger USDT amounts receive better FCC/USDT ratios, but this system can be exploited through order aggregation attacks.

```

function calculateFccByUsdt(uint256 _usdtAmount) internal pure returns (uint256) {
    if (_usdtAmount >= 100_000 * usdtDecimal) { // Tier 1: 1 FCC = 0.06 USDT
        return (_usdtAmount * 100 * fccDecimal) / (6 * usdtDecimal);
    } else if (_usdtAmount < 100_000 * usdtDecimal && _usdtAmount >= 10_000 * usdtDecimal) { // Tier 2:
1 FCC = 0.07 USDT
        return (_usdtAmount * 100 * fccDecimal) / (7 * usdtDecimal);
    } else if (_usdtAmount < 10_000 * usdtDecimal && _usdtAmount >= 5_000 * usdtDecimal) { // Tier 3: 1 F
FCC = 0.08 USDT
        return (_usdtAmount * 100 * fccDecimal) / (8 * usdtDecimal);
    } else if (_usdtAmount < 5_000 * usdtDecimal && _usdtAmount >= 1_000 * usdtDecimal) { // Tier 4: 1 F
CC = 0.09 USDT
        return (_usdtAmount * 100 * fccDecimal) / (9 * usdtDecimal);
    } else if (_usdtAmount < 1_000 * usdtDecimal && _usdtAmount > 0 * usdtDecimal) { // Tier 5: 1 FCC =
0.1 USDT
        return (_usdtAmount * 10 * fccDecimal) / usdtDecimal;
    } else {
        revert NotSupportUsdtAmount();
    }
}

```

1. Aggregator Contract: An attacker deploys a contract that collects USDT from multiple users
2. Order Collection: The contract accepts small USDT amounts (e.g., 500 USDT each) from many users
3. Aggregation: The contract aggregates all collected USDT into a single large amount (e.g., 100,000+ USDT)
4. Better Pricing: The aggregated amount qualifies for Tier 1 pricing (0.06 USDT per FCC) instead of Tier 4/5
5. Distribution: The contract distributes the FCC tokens back to users at the better rate

Example Attack:

1. 200 users each provide 500 USDT (Tier 4: 0.09 USDT/FCC)
2. Aggregator collects 100,000 USDT total
3. Aggregator buys FCC at Tier 1 rate (0.06 USDT/FCC)
4. Users get ~50% more FCC than they would individually

## Recommendation

\*\*\*: Limit the user to EOA. `tx.origin = msg.sender && msg.sender.code.length = 0`

## Client Response

client response : Fixed.

# FIC-11: Incorrect `_dropNumber` validation allows increasing mined rewards

| Category | Severity | Client Response | Contributor |
|----------|----------|-----------------|-------------|
| Logical  | Medium   | Fixed           | ***         |

## Code Reference

- code/src/contracts/core/FishcakeEventManagerV2.sol#L75
- code/src/contracts/core/FishcakeEventManagerV2.sol#L173-L178
- code/src/contracts/core/FishcakeEventManagerV2.sol#L226

```
75: _dropNumber <= 101 || _dropNumber <= _totalDropAmts / 10e6,
```

```
173: uint256 tmpDroppedVal = aie.alreadyDropNumber * 20 * 1e6;
174:         uint256 tmpBusinessMinedAmt =
175:             ((aie.alreadyDropAmts > tmpDroppedVal ? tmpDroppedVal : aie.alreadyDropAmts) * percent) / 100;
176:         if (tmpBusinessMinedAmt > maxMineAmtLimit) {
177:             tmpBusinessMinedAmt = maxMineAmtLimit;
178:         }
```

```
226: require(ai.dropNumber > aie.alreadyDropNumber, "FishcakeEventManager drop: Exceeded the number of rewards.");
```

## Description

\*\*\*: The `activityAdd()` function incorrectly validates `_dropNumber`, allowing up to `101` drops instead of the intended `100`. This affects the `activityFinish()` function's mining reward calculation, enabling users to receive higher mined amounts (`tmpBusinessMinedAmt`) than intended.

Incorrect validation in `activityAdd()`:

```
require(
>>     _dropNumber <= 101 || _dropNumber <= _totalDropAmts / 10e6, // Should be <= 100
        "FishcakeEventManager activityAdd: Drop Number Too Large, Limit 100 or TotalDropAmts/10."
);
```

The check `_dropNumber <= 101` allows one extra drop beyond the intended limit (`100`).

This affects the `drop()` function:

```
>> require(ai.dropNumber > aie.alreadyDropNumber, "FishcakeEventManager drop: Exceeded the number of rewards.");
```

Since `dropNumber` can be `101`, the function will allow up to `101` drops (when `alreadyDropNumber` reaches `100`, it still passes the check).

In `activityFinish()`, this impacts the mining calculation:

```
>> uint256 tmpDroppedVal = aie.alreadyDropNumber * 20 * 1e6; // Increases with more drops
uint256 tmpBusinessMinedAmt = ((aie.alreadyDropAmts > tmpDroppedVal ? tmpDroppedVal : aie.alreadyDropAmts) * percent) / 100;
```

Since `alreadyDropNumber` can reach `101`, `tmpDroppedVal` increases, leading to higher `tmpBusinessMinedAmt`.

## Recommendation

\*\*\*: Fix the validation in `activityAdd()`

```
require(
-  _dropNumber <= 101 || _dropNumber <= _totalDropAmts / 10e6,
+  _dropNumber < 101 || _dropNumber <= _totalDropAmts / 10e6,
  "FishcakeEventManager activityAdd: Drop Number Too Large, Limit 100 or TotalDropAmts/10."
);
```

## Client Response

client response : Fixed.



# FIC-12: Inconsistent Token Burn Tracking Due to Multiple Burn Mechanisms

| Category | Severity | Client Response | Contributor |
|----------|----------|-----------------|-------------|
| Logical  | Medium   | Fixed           | ***         |

## Code Reference

- code/src/contracts/core/token/FishCakeCoin.sol#L13
- code/src/contracts/core/token/FishCakeCoin.sol#L66-L70

```
13: contract FishCakeCoin is Initializable, ERC20Upgradeable, ERC20BurnableUpgradeable, OwnableUpgradeable, ReentrancyGuardUpgradeable, FishCakeCoinStorage {
```

```
66: function burn(address user, uint256 _amount) external onlyRedemptionPool {
67:     _burn(user, _amount);
68:     _burnedTokens += _amount;
69:     emit Burn(_amount, totalSupply());
70: }
```

## Description

\*\*\*: The contract inherits from `ERC20BurnableUpgradeable`, which provides two standard burn functions(`burn` and `burnFrom`):

```
contract FishCakeCoin is Initializable, ERC20Upgradeable, ERC20BurnableUpgradeable, OwnableUpgradeable, ReentrancyGuardUpgradeable, FishCakeCoinStorage {
    // Inherits burn() and burnFrom() from ERC20BurnableUpgradeable
}
```

However, the contract implements a custom `burn` tracking mechanism that only accounts for burns through the custom `burn()` function:

```
function burn(address user, uint256 _amount) external onlyRedemptionPool {
    _burn(user, _amount);
    _burnedTokens += _amount; // Only tracks custom burns
    emit Burn(_amount, totalSupply());
}
```

The `_burnedTokens` variable is meant to track total burned tokens:

```
uint256 public _burnedTokens; // Intended to track total burned tokens
```

However, Users can still call the inherited `burn()` and `burnFrom()` functions from `ERC20BurnableUpgradeable`, which will burn tokens but not update `_burnedTokens`. This creates a discrepancy where:

- `_burnedTokens` only tracks burns through the custom `burn()` function

- Actual total burned tokens = `_burnedTokens` + tokens burned via inherited functions

## Recommendation

\*\*\*: - Option 1: Override the inherited burn functions to track all burns:

- Option 2: Rename the variable to be more specific:

```
uint256 public _redemptionPoolBurnedTokens; // Only tracks redemption pool burns
```

- Option 3: Remove the custom tracking and rely on `totalSupply()` vs `MaxTotalSupply` to calculate burned tokens.

## Client Response

client response : Fixed. `_burnedTokens` changed to be `_redemptionPoolBurnedTokens`

# FIC-13: Improper use of `initializer` modifier in internal storage initialization functions

| Category | Severity | Client Response | Contributor |
|----------|----------|-----------------|-------------|
| Logical  | Medium   | Fixed           | ***         |

## Code Reference

- code/src/contracts/core/FishcakeEventManagerStorage.sol#L70

```
70: function __FishcakeEventManagerStorage_init(address _fccAddress, address _usdtTokenAddr, address _NFTManagerAddr) internal initializer {
```

- code/src/contracts/core/sale/DirectSalePoolStorage.sol#L20

```
20: function __DirectSalePoolStorage_init(address _fishCakeCoin, address _redemptionPool, address _tokenUsdtAddress) internal initializer {
```

- code/src/contracts/core/sale/InvestorSalePoolStorage.sol#L27

```
27: function __InvestorSalePoolStorage_init(address _fishCakeCoin, address _redemptionPool, address _tokenUsdtAddress) internal initializer {
```

- code/src/contracts/core/token/NftManagerStorage.sol#L63

```
63: function __NftManagerStorage_init(address _fccTokenAddr, address _tokenUsdtAddr, address _redemptionPoolAddress) internal initializer {
```

## Description

\*\*\*: Multiple internal storage contracts—specifically;

- `FishcakeEventManagerStorage`
- `NftManagerStorage`
- `DirectSalePoolStorage`
- `InvestorSalePoolStorage`

declare their internal initialization functions using the `initializer` modifier:

```
>> function __NftManagerStorage_init(address _fccTokenAddr, address _tokenUsdtAddr, address _redemptionPool
Address) internal initializer {
    fccTokenAddr = IERC20(_fccTokenAddr);
    tokenUsdtAddr = IERC20(_tokenUsdtAddr);
    redemptionPoolAddress = IRedemptionPool(_redemptionPoolAddress);

    _nextTokenId = 1;
    merchantValue = 8e7;
    userValue = 8e6;
    minedAmt = 0;

    proNftJson = "https://www.fishcake.org/image/1.json";
    basicNftJson = "https://www.fishcake.org/image/2.json";
}
```

This is incorrect. The `initializer` modifier is intended for top-level external initialization functions. Using it on internal `__init` functions can block chained initializations during contract upgrades or inheritance, as only one `initializer` can be executed per upgrade call context.

## Recommendation

\*\*\*: Change the `initializer` modifier to `onlyInitializing` in all internal `__init` functions to properly allow initializer chaining while still protecting from re-initialization outside the constructor context.

```
- function __NftManagerStorage_init(...) internal initializer
+ function __NftManagerStorage_init(...) internal onlyInitializing
```

Repeat the same fix for:

```
__FishcakeEventManagerStorage_init
__DirectSalePoolStorage_init
__InvestorSalePoolStorage_init
```

## Client Response

client response : Fixed.

# FIC-14: Hardcoded USDT Decimal Value Leading to Cross-Chain Incompatibility

| Category | Severity | Client Response | Contributor |
|----------|----------|-----------------|-------------|
| Logical  | Medium   | Acknowledged    | ***         |

## Code Reference

- code/src/contracts/core/sale/InvestorSalePool.sol#L92-L126

```
92: function calculateFccByUsdt(uint256 _usdtAmount) internal pure returns (uint256) {
93:     if (_usdtAmount >= 100_000 * usdtDecimal) { // Tier 1: 1 FCC = 0.06 USDT
94:         return (_usdtAmount * 100 * fccDecimal) / (6 * usdtDecimal);
95:     } else if (_usdtAmount < 100_000 * usdtDecimal && _usdtAmount >= 10_000 * usdtDecimal) { // Tier
2: 1 FCC = 0.07 USDT
96:         return (_usdtAmount * 100 * fccDecimal) / (7 * usdtDecimal);
97:     } else if (_usdtAmount < 10_000 * usdtDecimal && _usdtAmount >= 5_000 * usdtDecimal) { // Tier 3:
1 FCC = 0.08 USDT
98:         return (_usdtAmount * 100 * fccDecimal) / (8 * usdtDecimal);
99:     } else if (_usdtAmount < 5_000 * usdtDecimal && _usdtAmount >= 1_000 * usdtDecimal) { // Tier 4: 1
FCC = 0.09 USDT
100:         return (_usdtAmount * 100 * fccDecimal) / (9 * usdtDecimal);
101:     } else if (_usdtAmount < 1_000 * usdtDecimal && _usdtAmount > 0 * usdtDecimal) { // Tier 5: 1 FCC
= 0.1 USDT
102:         return (_usdtAmount * 10 * fccDecimal) / usdtDecimal;
103:     } else {
104:         revert NotSupportUsdtAmount();
105:     }
106: }
107:
108: function calculateUsdtByFccExternal(uint256 _amount) external pure returns (uint256) {
109:     return calculateUsdtByFcc(_amount);
110: }
111:
```

```
112: function calculateUsdtByFcc(uint256 _fccAmount) internal pure returns (uint256) {
113:     if (_fccAmount >= 5_000_000 * fccDecimal) { // tier1: 1 FCC = 0.06 USDT
114:         return (_fccAmount * 6 * usdtDecimal) / (100 * fccDecimal);
115:     } else if (_fccAmount < 5_000_000 * fccDecimal && _fccAmount >= 250_000 * fccDecimal) { // tier2:
1 FCC = 0.07 USDT
116:         return (_fccAmount * 7 * usdtDecimal) / (100 * fccDecimal);
117:     } else if (_fccAmount < 250_000 * fccDecimal && _fccAmount >= 100_000 * fccDecimal) { // tier3: 1
FCC = 0.08 USDT
118:         return (_fccAmount * 8 * usdtDecimal) / (100 * fccDecimal);
119:     } else if (_fccAmount < 100_000 * fccDecimal && _fccAmount >= 16_666 * fccDecimal) { // tier4: 1
FCC = 0.09 USDT
120:         return (_fccAmount * 9 * usdtDecimal) / (100 * fccDecimal);
121:     } else if (_fccAmount < 16_666 * fccDecimal && _fccAmount > 0 * fccDecimal) { // tier5: 1 FCC =
0.1 USDT
122:         return (_fccAmount * 10 * usdtDecimal) / (100 * fccDecimal);
123:     } else {
124:         revert NotSupportFccAmount();
125:     }
126: }
```

- code/src/contracts/core/sale/InvestorSalePoolStorage.sol#L14

```
14: uint256 public constant usdtDecimal = 10 ** 6;
```

## Description

\*\*\*: ### Vulnerability Description:

The `InvestorSalePoolStorage` contract contains a critical vulnerability due to **hardcoding USDT's decimal value as 6** ( $10^6$ ).

**USDT Decimals Vary Across Chains:**

- **Ethereum, Arbitrum, etc.:** USDT uses **6 decimals** (standard).
- **BSC, Polygon, etc.:** USDT uses **18 decimals** (common on EVM-compatible chains).

As a result, on chains where USDT has **18 decimals**, the contract will **overestimate USDT amounts** by a factor of  $10^{12}$  ( $1e18$  vs  $1e6$ ).

Example:

- A user sends **1 USDT ( $1e18$ )** on BSC.
- Contract interprets it as  **$1e12$  USDT ( $1e6$ )** due to hardcoded decimals.

**Financial Losses:**

- Users receive **far more FCC tokens than expected**.
- Protocol may **over-issue FCC** if decimals are misinterpreted.

**Example Attack Scenario:**

- On **BSC (USDT = 18 decimals)**, a user buys FCC with **1 USDT ( $1e18$ )**.
- Contract treats it as  $1e12$  USDT( $1e6$ ), User receives  $1e12$  times of the expected FCC.

## Recommendation

\*\*\*: Replace the hardcoded `usdtDecimal` with **dynamic decimal fetching** using `IERC20Metadata`:

Example:

```
import "@openzeppelin/contracts/token/ERC20/extensions/IERC20Metadata.sol";

abstract contract InvestorSalePoolStorage is Initializable, IInvestorSalePool {
    using SafeERC20 for IERC20;

    uint256 public fccDecimal = 10 ** 6; // FCC is fixed at 6 decimals
    uint256 public usdtDecimal;          // USDT decimals are set at initialization

    function __InvestorSalePoolStorage_init(
        address _fishCakeCoin,
        address _redemptionPool,
        address _tokenUsdtAddress
    ) internal initializer {
        fishCakeCoin = IERC20(_fishCakeCoin);
        redemptionPool = IRedemptionPool(_redemptionPool);
        tokenUsdtAddress = IERC20(_tokenUsdtAddress);
        usdtDecimal = 10 ** IERC20Metadata(_tokenUsdtAddress).decimals(); // Dynamic decimals
    }
}
```

## Client Response

client response : Acknowledged. Problem noted but would not be fixed for now.

# FIC-15: DepositIntoStaking function does not verify whether the Booster NFT exists.

| Category | Severity | Client Response | Contributor |
|----------|----------|-----------------|-------------|
| Logical  | Medium   | Fixed           | ***         |

## Code Reference

- code/src/contracts/core/StakingManager.sol#L39-78

```
39: function DepositIntoStaking(uint256 amount, uint8 stakingType) external nonReentrant {
40:     require(amount > minStakeAmount, "StakingManager DepositIntoStaking: staking amount must be more t
han minStakeAmount");
41:
42:     IERC20(fccAddress).safeTransfer(address(this), amount);
43:
44:     bytes32 txMessageHash = keccak256(
45:         abi.encode(
46:             msg.sender,
47:             fccAddress,
48:             amount,
49:             messageNonce
50:         )
51:     );
52:     uint256 stakingTimestamp = 0;
53:     uint256 apr = 0;
54:     (stakingTimestamp, apr) = getStakingPeriodAndApr(stakingType);
55:     uint endTime = block.timestamp + stakingTimestamp;
56:
57:     uint256 tokenId = nftManagerAddress.getActiveMinerBoosterNft(msg.sender);
58:
```

```
59:     stakeHolderStakingInfo memory ssInfo = stakeHolderStakingInfo({
60:         startStakingTime: block.timestamp,
61:         amount: amount,
62:         messageNonce: messageNonce,
63:         endStakingTime: endTime,
64:         stakingStatus: 0, // under staking now
65:         stakingType: stakingType,
66:         bindingNft: tokenId
67:     });
68:
69:     nftManagerAddress.inActiveMinerBoosterNft(msg.sender);
70:
71:     stakingQueued[msg.sender][txMessageHash] = ssInfo;
72:
73:     totalStakingAmount += amount;
74:
75:     emit StakeHolderDepositStaking(msg.sender, amount, messageNonce);
76:
77:     messageNonce++;
78: }
```

## Description



\*\*\*: In the DepositIntoStaking function, after retrieving the Booster NFT tokenId via `nftManagerAddress.getActiveMinerBoosterNft`, the contract does not validate whether the tokenId is non-zero. If a user has never minted a Booster NFT, the returned tokenId defaults to 0, allowing such users to proceed with staking and receive the base APR reward (1%).

## Recommendation

\*\*\*: Add a validation check in the DepositIntoStaking function to ensure that the user possesses a valid Booster NFT by verifying that the tokenId is non-zero before proceeding with staking logic.

## Client Response

client response : Fixed. function `getNftApr` is fixed which considers the situation `tokenId = 0`

# FIC-16:Any ETH received on stakingManager would locked permanently.

| Category          | Severity | Client Response | Contributor |
|-------------------|----------|-----------------|-------------|
| Language Specific | Medium   | Fixed           | ***         |

## Code Reference

- code/src/contracts/core/StakingManager.sol#L22C5-L24C6

```
NaN: receive() external payable {
NaN:     emit Received(msg.sender, msg.value);
NaN: }
```

## Description

\*\*\*: while there's a receive ETH mechanism implemented on stakingManager.

```
receive() external payable {
    emit Received(msg.sender, msg.value);
}
```

The issue is that there's no corresponding mechanism to extract such funds leading to ETH locking tendencies.

## Impact

Any ETH received on stakingManager would be stuck.

## Recommendation

\*\*\*: if intentions are to receive ETH implement the corresponding withdrawal mechanism.

## Client Response

client response : Fixed.

# FIC-17:Uninitialized Economic Parameters Allow Free NFT Minting

| Category | Severity | Client Response | Contributor |
|----------|----------|-----------------|-------------|
| Logical  | Low      | Fixed           | ***         |

## Code Reference

- code/src/contracts/core/token/NftManagerV5.sol#L79-L91

```
79: function initialize(  
80:     address _initialOwner,  
81:     address _fccTokenAddr,  
82:     address _tokenUsdtAddr,  
83:     address _redemptionPoolAddress  
84: ) public initializer {  
85:     require(_initialOwner != address(0), "NftManager initialize: _initialOwner can't be zero address");  
86:     __ERC721_init("Fishcake Pass NFT", "FNFT");  
87:     __ReentrancyGuard_init();  
88:     __Ownable_init(_initialOwner);  
89:     _transferOwnership(_initialOwner);  
90:     __NftManagerStorage_init(_fccTokenAddr, _tokenUsdtAddr, _redemptionPoolAddress);  
91: }
```

## Description

\*\*\*: The `initialize` function does not set `merchantValue`, `userValue`, or `validTime`, leaving them at their default value of `0`; because `createNFT` uses these uninitialized values to calculate `payUsdtAmount = (_type == 1 ? merchantValue : userValue)` and `nftDeadline = block.timestamp + validTime`, a caller can invoke `createNFT(..., _type)` before the owner calls `setValues` and `setValidTime` and receive an NFT for free (zero USDT transferred, since `transferFrom` of `0` succeeds) with a deadline equal to the current block timestamp. An attacker can mint an unlimited number of NFTs at no cost, completely bypassing the intended payment mechanism.

## Recommendation

\*\*\*: In the `initialize` function, explicitly set `merchantValue`, `userValue`, and `validTime` to nonzero defaults or require in `createNFT` that these parameters are nonzero before proceeding.

## Client Response

client response : Fixed.

## FIC-18: The `InvestorSalePool` contract lacks function to withdraw FCC tokens from the contract.

| Category | Severity | Client Response | Contributor |
|----------|----------|-----------------|-------------|
| Logical  | Low      | Fixed           | ***         |

### Code Reference

- code/src/contracts/core/sale/InvestorSalePool.sol#L14

```
14: contract InvestorSalePool is Initializable, ERC20Upgradeable, ERC20BurnableUpgradeable, OwnableUpgradeable, ReentrancyGuardUpgradeable, InvestorSalePoolStorage {
```

### Description

\*\*\*: The `InvestorSalePool` contract implements a token sale mechanism for Fishcake Coin (FCC) using `USDT` as payment. While the contract properly handles `USDT` withdrawals through the `withdrawUsdt()` function:

```
function withdrawUsdt(uint256 _amount) external onlyOwner {
    tokenUsdtAddress.transfer(vaultAddress, _amount);
    emit WithdrawUsdt(vaultAddress, _amount);
}
```

it lacks equivalent functionality to withdraw FCC tokens from the contract. This creates a potential risk of FCC tokens becoming permanently locked in the contract with no recovery mechanism.

### Recommendation

\*\*\*: Consider following fix:

```
function withdrawFcc(uint256 _amount) external onlyOwner nonReentrant {
    require(fishCakeCoin.balanceOf(address(this)) >= _amount, "Insufficient FCC balance");
    fishCakeCoin.transfer(vaultAddress, _amount);
    emit WithdrawFcc(vaultAddress, _amount);
}
```

### Client Response

client response : Fixed.

# FIC-19: Inconsistent Error Message in User Type Validation Leading to Logic Confusion

| Category | Severity | Client Response | Contributor |
|----------|----------|-----------------|-------------|
| Logical  | Low      | Fixed           | ***         |

## Code Reference

- code/src/contracts/core/token/NftManagerV5.sol#L130-L182

```
130: function createNFT(
131:     string memory _businessName,
132:     string memory _description,
133:     string memory _imgUrl,
134:     string memory _businessAddress,
135:     string memory _website,
136:     string memory _social,
137:     uint8 _type
138: ) external nonReentrant returns (bool, uint256) {
139:     require(
140:         _type == 1 || _type == 2,
141:         "NftManager createNFT: type can only equal 1 and 2, 1 stand for merchant, 2 stand for persona
1 user"
142:     );
143:     uint256 payUsdtAmount = _type == 1 ? merchantValue : userValue;
144:     uint256 nftDeadline = block.timestamp + validTime;
145:     if (_type == 1) {
146:         require(
147:             tokenUsdtAddr.allowance(msg.sender, address(this)) >= merchantValue,
148:             "NftManager createNFT: Merchant allowance must more than 80 U"
149:         );
```

```
150:         merchantNftDeadline[msg.sender] = nftDeadline;
151:         fccTokenAddr.transfer(msg.sender, proMineAmt);
152:     } else {
153:         require(
154:             tokenUsdtAddr.allowance(msg.sender, address(this)) >= userValue,
155:             "NftManager createNFT: Merchant allowance must more than 8 U"
156:         );
157:         userNftDeadline[msg.sender] = nftDeadline;
158:         fccTokenAddr.transfer(msg.sender, basicMineAmt);
159:     }
160:
161:     tokenUsdtAddr.transferFrom(msg.sender, address(this), payUsdtAmount);
162:     tokenUsdtAddr.transfer(address(redemptionPoolAddress), (payUsdtAmount * 75) / 100);
163:
164:     uint256 tokenId = _nextTokenId++;
165:     _safeMint(msg.sender, tokenId);
166:     nftMintType[tokenId] = _type;
167:
168:     emit CreateNFT(
169:         msg.sender,
```

```
170:         tokenId,  
171:         _businessName,  
172:         _description,  
173:         _imgUrl,  
174:         _businessAddress,  
175:         _website,  
176:         _social,  
177:         payUsdtAmount,  
178:         nftDeadline,  
179:         _type  
180:     );  
181:     return (true, tokenId);  
182: }
```

## Description

\*\*\*: in the `createNFT` function across all versions of the NFT Manager contracts. The issue manifests as an incorrect error message displayed when user type validation fails for personal users (`_type == 2`). Despite the validation logic correctly checking user-specific values, the error message incorrectly references "Merchant allowance" instead of "User allowance", creating a logical inconsistency between the validation logic and user feedback.

### Technical Root Cause:

The vulnerability stems from copy-paste programming where the merchant error message was duplicated for user validation without proper modification:

```
// Correct merchant validation (type == 1)  
if (_type == 1) {  
    require(  
        tokenUsdtAddr.allowance(msg.sender, address(this)) >= merchantValue,  
        "NftManager createNFT: Merchant allowance must more than 80 U" // ✅ CORRECT  
    );  
    merchantNftDeadline[msg.sender] = nftDeadline;  
    fccTokenAddr.transfer(msg.sender, proMineAmt);  
} else {  
    // Incorrect user validation (type == 2)  
    require(  
        tokenUsdtAddr.allowance(msg.sender, address(this)) >= userValue,  
        "NftManager createNFT: Merchant allowance must more than 8 U" // ❌ INCORRECT  
    );  
    userNftDeadline[msg.sender] = nftDeadline;  
    fccTokenAddr.transfer(msg.sender, basicMineAmt);  
}
```

### Exploitation Scenario:

#### Step 1: Setup Conditions

```
// User has insufficient USDT allowance for personal NFT creation
// userValue = 8 USDT (typically)
// User's current allowance < 8 USDT
address user = 0x123...;
uint256 currentAllowance = usdtToken.allowance(user, nftManagerAddress);
require(currentAllowance < userValue, "User must have insufficient allowance");
```

## Step 2: Trigger the Inconsistency

```
// User attempts to create personal NFT with insufficient allowance
vm.startPrank(user);
vm.expectRevert("NftManager createNFT: Merchant allowance must more than 8 U");

nftManager.createNFT(
    "Personal Business",
    "My personal NFT",
    "https://example.com/image.png",
    "123 Main St",
    "https://mybusiness.com",
    "@mybusiness",
    2 // Personal user type
);
vm.stopPrank();
```

## Step 3: Observe Inconsistent Behavior

```
// Expected vs Actual Error Messages:
// ACTUAL (Incorrect): "NftManager createNFT: Merchant allowance must more than 8 U"
// EXPECTED (Correct): "NftManager createNFT: User allowance must more than 8 U"

// The error message incorrectly suggests a merchant issue when:
// 1. User is creating a personal NFT (type == 2)
// 2. Validation is checking userValue, not merchantValue
// 3. The logic path is in the 'else' branch for non-merchant users
```

## Recommendation

\*\*\*: Update the error message in the user validation branch to accurately reflect the validation context:

```
// BEFORE (Incorrect):
} else {
    require(
        tokenUsdtAddr.allowance(msg.sender, address(this)) >= userValue,
        "NftManager createNFT: Merchant allowance must more than 8 U" // ❌ WRONG
    );
    userNftDeadline[msg.sender] = nftDeadline;
    fccTokenAddr.transfer(msg.sender, basicMineAmt);
}

// AFTER (Correct):
} else {
    require(
        tokenUsdtAddr.allowance(msg.sender, address(this)) >= userValue,
        "NftManager createNFT: User allowance must more than 8 U" // ✅ CORRECT
    );
    userNftDeadline[msg.sender] = nftDeadline;
    fccTokenAddr.transfer(msg.sender, basicMineAmt);
}
```

**Enhanced Solution with Constants:**



```
// Define error message constants for consistency
string private constant MERCHANT_ALLOWANCE_ERROR = "NftManager createNFT: Merchant allowance must more than 80 U";
string private constant USER_ALLOWANCE_ERROR = "NftManager createNFT: User allowance must more than 8 U";

function createNFT(..., uint8 _type) external nonReentrant returns (bool, uint256) {
    require(_type == 1 || _type == 2, "Invalid type");

    uint256 payUsdtAmount = _type == 1 ? merchantValue : userValue;
    uint256 nftDeadline = block.timestamp + validTime;

    if (_type == 1) {
        require(
            tokenUsdtAddr.allowance(msg.sender, address(this)) >= merchantValue,
            MERCHANT_ALLOWANCE_ERROR
        );
        merchantNftDeadline[msg.sender] = nftDeadline;
        fccTokenAddr.transfer(msg.sender, proMineAmt);
    } else {
        require(
            tokenUsdtAddr.allowance(msg.sender, address(this)) >= userValue,
            USER_ALLOWANCE_ERROR
        );
        userNftDeadline[msg.sender] = nftDeadline;
        fccTokenAddr.transfer(msg.sender, basicMineAmt);
    }

    // ... rest of function
}
```

## Client Response

client response : Fixed.

# FIC-20:Redundant Code

| Category | Severity      | Client Response | Contributor |
|----------|---------------|-----------------|-------------|
| Logical  | Informational | Fixed           | ***         |

## Code Reference

- code/src/contracts/core/sale/DirectSalePoolV1.sol#L32

```
32: __ERC20_init("FishCake", "FCC");
```

- code/src/contracts/core/sale/InvestorSalePool.sol#L14

```
14: contract InvestorSalePool is Initializable, ERC20Upgradeable, ERC20BurnableUpgradeable, OwnableUpgradeable, ReentrancyGuardUpgradeable, InvestorSalePoolStorage {
```

- code/src/contracts/core/StakingManager.sol#L97
- code/src/contracts/core/StakingManager.sol#L153-L154

```
97: if (amountOut < minStakeAmount) {
```

```
153: uint256 decimal = 10e6;  
154:         uint8 nftType = nftManagerAddress.getMinedBoosterNftType(tokenId);
```

## Description

\*\*\*: In the `StakingManager.sol` contract, the `getNftApr` function contains an unused variable declaration:

```
function getNftApr(address miner, uint256 tokenId) internal view returns(uint256) {  
@=>    uint256 decimal = 10e6;  
    uint8 nftType = nftManagerAddress.getMinedBoosterNftType(tokenId);  
    if (nftType == 6) {  
        return 20;  
    } else if (nftType == 5) {  
        return 15;  
    } else if (nftType == 4) {  
        return 9;  
    } else if (nftType == 3) {  
        return 5;  
    } else {  
        return 1;  
    }  
}
```

The `decimal` variable is declared on line 152 but is never used anywhere in the function. The function simply returns hardcoded APR values based on the NFT type without any decimal calculations.

\*\*\*: In the `StakingManager` contract, there is a redundant validation check in the `withdrawFromStakingWithAprIncome` function:

```
uint256 amountOut = stakingQueued[msg.sender][txMessageHash].amount;
if (amountOut < minStakeAmount) {
    revert NoFundingForStaking();
}
```

The withdraw function checks if `amountOut < minStakeAmount`, but this check is logically impossible to fail because:

- `minStakeAmount` =  $10 * 10^6$
- The deposit function enforces `amount > minStakeAmount` before storing the stake
- Therefore, `amountOut` (the stored amount) will always be  $> \text{minStakeAmount}$
- The condition `amountOut < minStakeAmount` will never be true

\*\*\*: The `InvestorSalePool` contract unnecessarily inherits from `ERC20Upgradeable`, which increases gas consumption.

Why does `DirectSalePoolV1` inherit from `ERC20Upgradeable` and call `__ERC20_init("FishCake", "FCC")` in the `initialize` function? This also seems unnecessary.

## Recommendation

\*\*\*: Remove the unused `decimal` variable declaration.

\*\*\*: Remove the redundant check from the `withdrawFromStakingWithAprIncome` function.

\*\*\*: It is recommended to remove the related redundant code.

## Client Response

client response : Fixed.

# FIC-21:Missing storage gap for upgradeable contract

| Category | Severity      | Client Response | Contributor |
|----------|---------------|-----------------|-------------|
| Logical  | Informational | Fixed           | ***         |

## Code Reference

- code/src/contracts/core/FishcakeEventManagerStorage.sol#L10

```
10: abstract contract FishcakeEventManagerStorage is Initializable, IFishcakeEventManager {
```

## Description

\*\*\*:

stakingManagerStorage is missing storage gap. This is important for upgradeable contracts to avoid storage collision.

## Recommendation

\*\*\*: Include storage gap at the end as in other upgradeable contract.

```
uint256[50] private __gap;
```

## Client Response

client response : Fixed.

# FIC-22:Missing ReentrancyGuardUpgradeable Initialization

| Category | Severity      | Client Response | Contributor |
|----------|---------------|-----------------|-------------|
| Logical  | Informational | Fixed           | ***         |

## Code Reference

- code/src/contracts/core/token/FishCakeCoin.sol#L26-L35

```
26: function initialize(address _owner, address _RedemptionPool) public initializer {
27:     require(_owner != address(0), "FishCakeCoin initialize: _owner can't be zero address");
28:     __ERC20_init(NAME, SYMBOL);
29:     __ERC20Burnable_init();
30:     __Ownable_init(_owner);
31:     RedemptionPool = _RedemptionPool;
32:     _transferOwnership(_owner);
33:     isAllocation = false;
34: }
```

## Description

\*\*\*: The contract inherits from multiple OpenZeppelin upgradeable contracts and properly initializes most of them in the initialize function:

```
function initialize(address _owner, address _RedemptionPool) public initializer {
    require(_owner != address(0), "FishCakeCoin initialize: _owner can't be zero address");
    __ERC20_init(NAME, SYMBOL);           // ✓ ERC20Upgradeable initialized
    __ERC20Burnable_init();               // ✓ ERC20BurnableUpgradeable initialized
    __Ownable_init(_owner);               // ✓ OwnableUpgradeable initialized
    RedemptionPool = _RedemptionPool;
    _transferOwnership(_owner);
    isAllocation = false;
}
```

However, the contract also inherits from `ReentrancyGuardUpgradeable` but missing the corresponding initialization call:

```
contract FishCakeCoin is Initializable, ERC20Upgradeable, ERC20BurnableUpgradeable, OwnableUpgradeable, ReentrancyGuardUpgradeable, FishCakeCoinStorage {
    // Missing: __ReentrancyGuard_init();
}
```

This means the reentrancy guard state variables are not properly initialized, this violates the best practice and could cause problems when changes are made to `ReentrancyGuardUpgradeable`.

## Recommendation

\*\*\*: Add the missing `__ReentrancyGuard_init()` call in the `initialize` function.

## Client Response

client response : Fixed.

# FIC-23:Missing Event

| Category   | Severity      | Client Response | Contributor |
|------------|---------------|-----------------|-------------|
| Code Style | Informational | Fixed           | ***         |

## Code Reference

- code/src/contracts/core/token/FishCakeCoin.sol#L46-55

```
46: RedemptionPool = _RedemptionPool;
47:     }
48:
49:     function setPoolAddress(fishCakePool memory _pool) external onlyOwner {
50:         _beforeAllocation();
51:         _beforePoolAddress(_pool);
52:         fcPool = _pool;
53:     }
54:
55:     function poolAllocate() external onlyOwner {
```

## Description

\*\*\*: Missing Event

The `setRedemptionPool` and `setPoolAddress` functions in the FishCakeCoin contract lack corresponding events. In Solidity, the absence of events for important state changes can lead to reduced transparency and make it difficult for off-chain services, such as blockchain explorers, monitoring tools, or dApps, to track and verify these changes. This can hinder auditing, debugging, and user trust.

## Recommendation

\*\*\*: It is recommended to emit events whenever critical contract parameters are updated.

## Client Response

client response : Fixed.

# FIC-24:Incorrect minimum stake validation

| Category | Severity      | Client Response | Contributor |
|----------|---------------|-----------------|-------------|
| Logical  | Informational | Fixed           | ***         |

## Code Reference

- code/src/contracts/core/StakingManager.sol#L40

```
40: require(amount > minStakeAmount, "StakingManager DepositIntoStaking: staking amount must be more than minStakeAmount");
```

## Description

\*\*\*: In the `DepositIntoStaking` function, the minimum stake amount is enforced using the following check;

```
>> require(amount > minStakeAmount, "StakingManager DepositIntoStaking: staking amount must be more than minStakeAmount");
```

This logic excludes users who attempt to stake exactly the `minStakeAmount`, even though the constant is defined as a valid minimum;

```
uint256 public constant minStakeAmount = 10 * 10 ** 6; // 10 FCC
```

Users expecting to stake the minimum amount are incorrectly blocked.

## Recommendation

\*\*\*: Modify

```
- require(amount > minStakeAmount, "StakingManager DepositIntoStaking: staking amount must be more than minStakeAmount");  
+ require(amount >= minStakeAmount, "StakingManager DepositIntoStaking: staking amount must be at least minStakeAmount");
```

## Client Response

client response : Fixed.



# FIC-25:Incorrect error message in `StakingManager::getStakingPeriodAndApr()`

| Category | Severity      | Client Response | Contributor |
|----------|---------------|-----------------|-------------|
| Logical  | Informational | Fixed           | ***         |

## Code Reference

- code/src/contracts/core/StakingManager.sol#L169

```
169: require(stakingType > 0 && stakingType < 5, "StakingManager getStakingPeriod: stakingType amount must be more than 0 and less than 4");
```

## Description

\*\*\*: In `StakingManager::getStakingPeriodAndApr()`, it is required that the provided `stakingType` meet the following condition:

```
require(stakingType > 0 && stakingType < 5, "StakingManager getStakingPeriod: stakingType amount must be more than 0 and less than 4");
```

However, the error message is misleading.

## Recommendation

\*\*\*: Modify:

```
- require(stakingType > 0 && stakingType < 5, "StakingManager getStakingPeriod: stakingType amount must be more than 0 and less than 4");  
+ require(stakingType > 0 && stakingType < 5, "StakingManager getStakingPeriod: stakingType amount must be more than 0 and less than 5");
```

## Client Response

client response : Fixed.

# FIC-26: Incorrect Upgradeable Imports Break Initialization Protection

| Category | Severity      | Client Response | Contributor |
|----------|---------------|-----------------|-------------|
| Logical  | Informational | Fixed           | ***         |

## Code Reference

- code/src/contracts/core/StakingManager.sol#L5-L7
- code/src/contracts/core/StakingManager.sol#L19

```
5: import "@openzeppelin-upgrades/contracts/proxy/utils/Initializable.sol";
6: import "@openzeppelin-upgrades/contracts/access/OwnableUpgradeable.sol";
7: import "@openzeppelin-upgrades/contracts/utils/ReentrancyGuardUpgradeable.sol";
```

```
19: _disableInitializers();
```

## Description

\*\*\*: Because the `_disableInitializers()` in the constructor only affects the implementation's own initializer state in the context of the incorrect import, not the proxy's, leaving the proxy's initializer guard ineffective. An attacker can re-initialize the proxy to become `owner`, enabling them to change critical addresses, drain funds, or arbitrarily upgrade the contract.

## Recommendation

\*\*\*: Switch all upgradeable imports to `@openzeppelin/contracts-upgradeable` and redeploy, ensuring `_disableInitializers()` is called on the implementation and `initialize` is protected by the official `initializer` modifier.

## Client Response

client response : Fixed.

# FIC-27:Function Naming Convention Violation

| Category | Severity      | Client Response | Contributor |
|----------|---------------|-----------------|-------------|
| Logical  | Informational | Fixed           | ***         |

## Code Reference

- code/src/contracts/core/StakingManager.sol#L39

```
39: function DepositIntoStaking(uint256 amount, uint8 stakingType) external nonReentrant {
```

## Description

\*\*\*: The `DepositIntoStaking` function violates Solidity naming conventions by using **PascalCase** instead of the standard **camelCase**, creating inconsistency with other functions in the contract.

```
function DepositIntoStaking(uint256 amount, uint8 stakingType) external nonReentrant {  
    ...  
}
```

## Recommendation

\*\*\*: Rename the function to follow camelCase convention:

```
function depositIntoStaking(uint256 amount, uint8 stakingType) external nonReentrant {
```

## Client Response

client response : Fixed.

# Disclaimer

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Invoices, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Invoice. This report provided in connection with the services set forth in the Invoices shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Invoice. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without Secure3's prior written consent in each instance.

This report is not an "endorsement" or "disapproval" of any particular project or team. This report is not an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Secure3 to perform a security assessment. This report does not provide any warranty or guarantee of free of bug of codes analyzed, nor do they provide any indication of the technologies, business model or legal compliancy.

This report should not be used in any way to make decisions around investment or involvement with any particular project. Instead, it represents an extensive assessing process intending to help our customers increase the quality of their code and high-level consistency of implementation and business model, while reducing the risk presented by cryptographic tokens and blockchain technology.

Secure3's position on the final decisions over blockchain technologies and corresponding associated transactions is that each company and individual are responsible for their own due diligence and continuous security.

The assessment services provided by Secure3 is subject to dependencies and under continuing development. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.